

WESTERN SYDNEY
UNIVERSITY



**Validating Efficiency of the
Forward-Forward Algorithm**

Nowell Kevin Reyes (20658133) and Gabriel Schussler
(20470601)

Final report for COMP3035 Discovery Project

Supervisor: Dr. Rosalind Wang

*School of Computer, Data and Mathematical Sciences,
Western Sydney University*

Spring, 2024

Contents

1	Abstract	3
2	Background	3
2.1	Introduction	3
2.2	Literature Review	4
3	Objective	5
4	Hypothesis/Research question	5
5	Methodology	5
5.1	Data	5
5.2	Tools/Code	6
5.3	Implementation	6
5.3.1	Test Accuracy Implementation	6
5.3.2	Epoch Control Implementation	7
5.3.3	Code to generate negative samples	7
6	Results & Discussions	8
6.1	Epoch control	8
6.2	Test accuracy control	12
6.3	Limitations	13
7	Future Research	13
8	Conclusion	14
9	Appendix	16

1 Abstract

This research project examines the computational efficiency of Geoffrey Hinton’s Forward-Forward algorithm relative to traditional backpropagation in artificial neural network training. The Forward-Forward (FF) algorithm replaces the usual forward-backward cycle with two forward passes using positive and negative data samples, which may improve training speed and resource efficiency. We implemented both the FF and backpropagation (BP) algorithms in PyTorch, comparing their performance on the MNIST dataset in terms of training time, GPU memory usage, GPU utilisation, and power consumption.

At a target accuracy of 97%, the FF algorithm reached this threshold approximately 50 seconds faster than BP and uses 20% less total power. Additionally, the FF method showed reduced GPU utilisation (27.39% vs 36.03%), indicating potential benefits in resource-limited settings, though memory usage was similar across methods. Although this study faced limitations related to hardware and implementation, the results suggest that the FF algorithm may provide valuable efficiency advantages, particularly in energy-sensitive contexts. Future research could explore the algorithm’s performance across other datasets, various network architectures, and applications beyond computer vision.

2 Background

2.1 Introduction

This section is written by Kevin

Artificial Neural Networks (NN) have made significant progress since the introduction of the Perceptron in the late 1950s. The work of Rumelhart et al. [1986] on the development of the backpropagation (BP) algorithm became the backbone of NN training and has since become the fundamental approach to training almost all NNs, from simple multilayer perceptron (MLP) [Rosenblatt, 1958], to deep NNs such as CNN [LeCun et al.], GAN [Goodfellow et al., 2014], and most recently large language models (LLMs) like ChatGPT [Naveed et al., 2024].

The strength of the BP algorithm lies in its ability to fine-tune the network’s parameters through an iterative process of forward and backward passes. In a forward pass, the input data moves through the network, generating predictions. A loss function then calculates the error between these predictions and the actual results. The gradient is then calculated with respect to the weights and biases from previous layers and is sent back through the network to adjust the connections up to the input layer [Rumelhart et al., 1986, Narayanan et al., 2021]. This repeated process, also known as epoch, enables the network to learn complex patterns from data.

However, as these NN models grow in size and complexity, the limitations of BP are becoming more evident. Neuroscientists also understand that the human brain does not optimise in the same way as backpropagation, at least not in the way machine learning models do [Lillicrap et al., 2020]. The brain does not have a single cost function from where the entirety of the brain weights are updated based on the gradient of that cost function. That would be akin to the brain stopping to function in *real time* to perform backpropagation, which is not plausible. On top of this, because BP requires “perfect knowledge of the computation performed in the forward pass to compute the correct derivatives” [Hinton]. If a black box is inserted

into the forward pass, we can no longer do BP unless we learn the differentiable model of the black box [Hinton].

The biological implausibility of BP has become one of the motivating factors that drove Geoffrey Hinton, one of the original founders of the BP algorithm, to propose a new learning method. The Forward-Forward (FF) algorithm supposedly does not require perfect knowledge of the forward pass and is therefore not impeded by black boxes or resort to computing gradients through backpropagation.

2.2 Literature Review

Published at the end of 2022, Hinton presented the Forward-Forward algorithm, its design motivated by the search for more biologically realistic learning processes, and the desire for more efficient, localised learning rules, to better model the human brain [Hinton, Bewtra, 2023].

Borrowing inspiration from Boltzmann Machines [Hinton and Sejnowski, 2001] and Noise-contrastive estimation [Gutmann and Hyvärinen, 2010], the FF algorithm replaces the traditional forward-backward cycle with two forward passes, each with its own opposing objectives. Hinton proposes training the model with two similar input data, the *positive* (i.e. real) data and *negative* data which can be put in manually or generated by the network itself [Hinton]. The form of negative data we used in this project will be covered in the Methodology section.

The positive pass’ objective is to ”adjust the weights to increase the goodness in every hidden layer” [Hinton], and for the negative pass to do the opposite. The *goodness* function here is simply the sum of squared neural activities of the hidden layers, and so the objectives of each passes is to maximise the activities of the neurons in the positive pass, and to decrease the activities of the neurons in the negative pass.

Each layer is trained in sequence to completion and is optimised by calculating the layer’s loss function as the goodness of a layer minus some *threshold* [Hinton]

$$p(x_i \in P) = \sigma(\sum_j y_j^2 - thresh)$$

where x_i is the positive input data, y_j the activity of hidden unit j before layer normalisation, and P the set of positive data.

When computing the loss with this objective function, FF only updates the weights connecting to its own hidden layer through gradient descent. [Amari, 1967]. Therefore, there is no need to backpropagate the gradients from the last layer to the input layer, potentially speeding up the training process. Once all the layers in the network are fully trained, we make a prediction for a test data by the collecting the *goodnesses* (i.e. neural activities) learned by the hidden layers and passing them to a Softmax function to classify the data.

Although the FF is still in its early stages, it offers an exciting new direction for research in NN training. Upon initial literature review, early studies have looked into different architectures [Paliotta et al., 2023, Scodellaro et al., 2024], hyperparameters [Gandhi et al., 2023], and potential applications (training on embedded devices with less computing resources) [Momeni et al., 2023, Reyes-Angulo and Paheding, 2023] of the algorithm, but a comparison of the FF and BP in terms of training speed and memory efficiency has yet to be done. This gap in the research field offers a valuable

opportunity that we'll explore with this project - to extend our knowledge of the literature, and upon reproduction of the `Python` implementation of the algorithm, we hope to contribute to the understanding of the Forward-Forward algorithm's practical advantages and limitations, pushing forward its potential applications in neural network training.

3 Objective

In this project, we aim to investigate the Forward-Forward algorithm, focusing on its performance in a supervised learning task, as described in Part 3.3 of Hinton's paper [Hinton, 2022].

Our first objective is to develop a sound understanding of the FF method by studying, reproducing and validating the `Python` implementation of the algorithm, to ensure its accurate representation of Hinton's version (originally written in MATLAB).

We then aim to quantify and compare the computational efficiency of the FF and BP by measuring training time and memory usage allocation for both algorithms using benchmark datasets and simple network architectures while maintaining similar accuracy results from the paper. This comparison will help determine which algorithm is more efficient in terms of computational resources and evaluate potential performance trade-offs against efficiency gains.

Finally, we will analyse our results in the context of practical applications, identifying scenarios where the FF might be preferable to BP (and vice versa), and outline potential directions for future research in this area.

This section is written by Kevin & Gabriel

4 Hypothesis/Research question

Does the Forward-Forward algorithm offer significant improvements in training time and/or memory efficiency compared to backpropagation for supervised classification tasks?

We expect the results from this project will provide a measurable benchmark for evaluating the FF algorithm's potential, as an alternative to BP, in terms of training time and memory usage for supervised classification tasks.

This section is written by Kevin

5 Methodology

5.1 Data

In this project, a popular dataset will be used to benchmark the efficiency of the Forward-Forward algorithm compared with the traditional backpropagation method in training image processing neural networks.

The MNIST data set, published by the National Institute of Standards and Technology [LeCun et al., 2010] consists of 60,000 training images and 10,000 testing images of handwritten digits converted to 28×28 pixel images. It is a commonly used dataset to benchmark the performance of image processing Neural Networks, hence why it has been chosen for this project.

This section is written by Gabriel

5.2 Tools/Code

The tools and code in the project will consist of open-source software and data to ensure that the project can be easily reproduced. The neural network models will be created using the Python library PyTorch, a widely used machine learning package.

Another Python library, `gpustats`¹, will be used to measure the usage of computing resources of each model. This library will give us the tools to measure several compute resources important to main objectives of this project, namely, power draw, RAM, GPU utilization and GPU memory.

The code for the neural networks will be sourced from other open-source projects online. This is done to ensure that implementation of the Forward-Forward algorithm is done correctly. Additionally, multiple LLMs (Copilot, ChatGPT-4o, and Claude’s Sonnet 3.5) were used in coding some of the more complex code for this project. All code for implementing FF and BP can be found on our GitHub².

Forward-Forward Algorithm Implementations The forward-forward algorithm code will be sourced from external code repositories as an in-house implementation of the forward-forward algorithm falls outside the scope of this report. In the preliminary research for this project, we found working implementations of the forward-forward algorithm in multiple code repositories.

The first implementation³ is a Pytorch script written by Sindy Löwex. While this implementation is sophisticated, and appears to be an accurate representation of the forward-forward algorithm, it may be difficult to implement alongside a traditional BP neural network such that the network architecture is comparable.

The second implementation⁴ is also a Pytorch script authored by Mohammed Pezeshki and Junho Yun. This version of the forward-forward algorithm is a simpler implementation and will integrate more seamlessly with the rest of code for metric tracking.

5.3 Implementation

Below is a break down of the overall structure of the steps used to produce the results of this project:

1. Build two neural networks using Pytorch with the same architecture. In the case of this report, a 3-layer network with 1000 neurons per layer was used. In the first network, use back propagation as the training algorithm and for the second network, use forward forward as the training algorithm.

5.3.1 Test Accuracy Implementation

2. Set a test accuracy threshold, in the case of this report 97% was chosen. Train each model until the test accuracy threshold is met. Track the following metrics across time using the GPUstats library:

- GPU Utilization

¹<https://github.com/wookayin/gpustat>

²https://github.com/Reinn90/FF_DP_2024

³https://github.com/loeweX/Forward-Forward/blob/main/src/ff_model.py

⁴https://github.com/mpezeshki/pytorch_forward_forward

- GPU Power Draw
 - GPU Memory Usage
3. Repeat Step 2 a minimum of 10 times due to the stochastic nature of training the models.
 4. Calculate the average value and standard deviation for each metric over time. Calculate the total power usage by multiplying the average power usage by the time each model took to train.

5.3.2 Epoch Control Implementation

5. Specify a number of epochs to train both models for, in the case of this report 50 epochs were chosen.
6. Train each model for 50 epochs. For the FF algorithm, record the layer loss for each layer, and for the BP algorithm, record the training loss and validation loss.
7. The ratio between FF and BP for each metric across each epoch and the data was bootstrapped to determine if there was a statistical difference.
8. Extract the weights for each layer for the model.

5.3.3 Code to generate negative samples

In the case of the Forward-Forward model, the one-hot encoding of the class will be appended to the beginning of each image. This will allow the model to train on positive and negative images required in the learning process of the FF algorithm, as shown in Figure 1⁵. In the case of the backpropagation, the image and class will be fed into the model as separate variables.

The negative samples are generated by randomly assigning a class label to each image, as shown in the function below:

```
def _get_neg_sample(self, sample, class_label):
    # Create randomly sampled one-hot label.
    classes = list(range(self.num_classes))
    classes.remove(class_label) # Remove true label from possible choices.
    wrong_class_label = np.random.choice(classes)
    wrong_class_label = torch.tensor(wrong_class_label, dtype=torch.long)
    one_hot_label = torch.nn.functional.one_hot(
        torch.tensor(wrong_class_label), num_classes=self.num_classes
    )
    neg_sample = sample.clone()
    neg_sample[:, 0, : self.num_classes] = one_hot_label
    return neg_sample
```

⁵Sourced from https://github.com/mpezeshki/pytorch_forward_forward

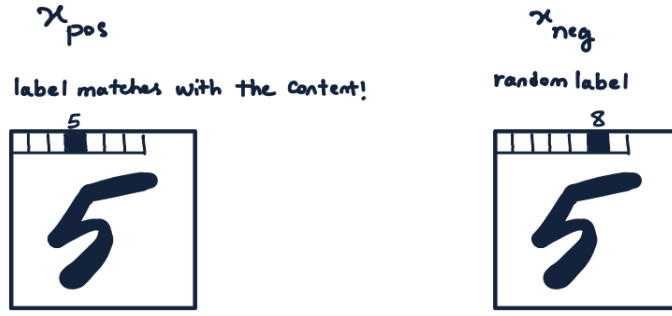


Figure 1: Visual depiction of positive and negative labels will be appended to each image.

6 Results & Discussions

6.1 Epoch control

We first ran both models using the same epochs as the control. This allowed us to test and gain an understanding of the accuracy and metrics the model will provide. We used 50 epochs for training both the BP and FF methods. As shown in Figure 2, the FF algorithm exhibits a similar curve as BP per layer, meaning the local objective functions are working as intended. Notably, in the FF model, we can see that each layer starts with a high loss at the beginning of a training epoch. This is due to the fact that the weights of each layer are initialised randomly, and since the network is trained in sequence (no back propagating gradients) the subsequent hidden layers start at a high loss value. We do see the loss decrease relatively quickly and plateau after around 10 epochs for FF.

This section is written by Kevin & Gabriel

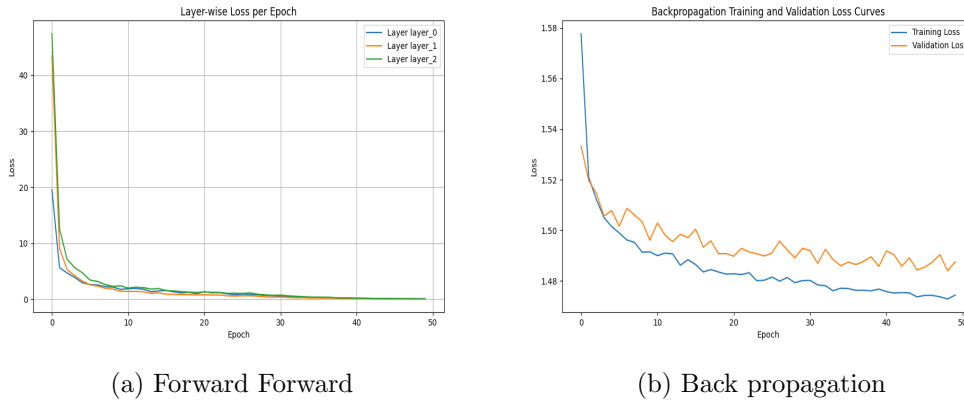


Figure 2: FF vs BP loss curves

A t-SNE visualisation [Van der Maaten and Hinton, 2008] of the layer embeddings confirms our expectations about the FF model's trained parameters. As shown in Figure 3, the model effectively learns the correct representation of the test data. This is further supported by the weight distributions of each layer being Gaussian

and centered around zero, indicating the algorithm is performing as expected (Figure 4).

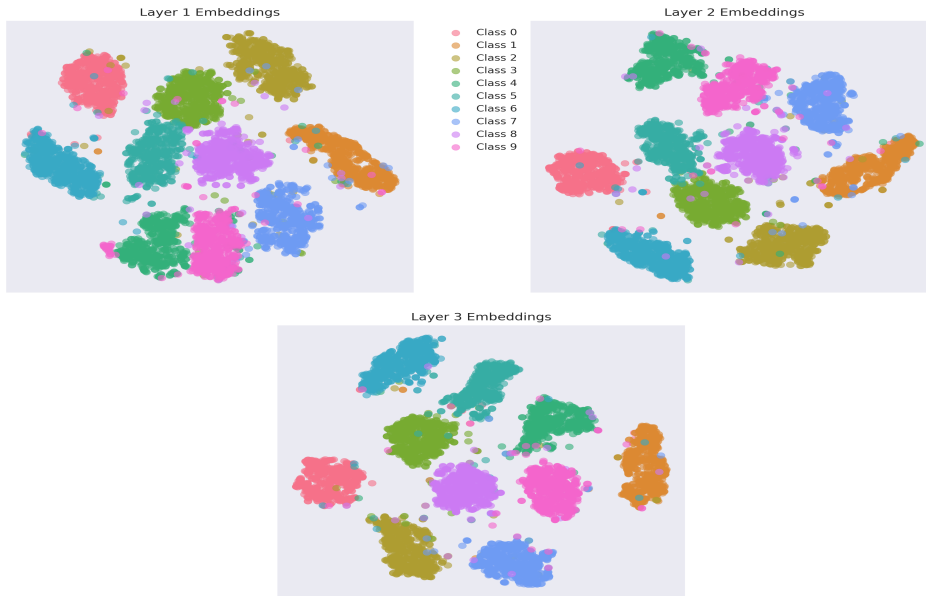
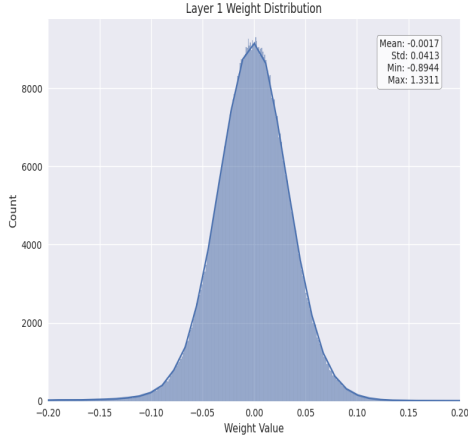
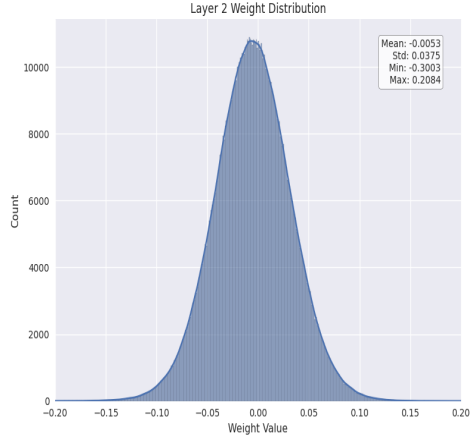


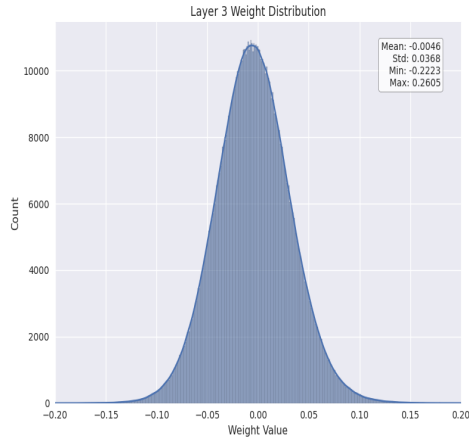
Figure 3: FF Layer embeddings t-SNE visualisation



(a) Layer 1



(b) Layer 2



(c) Layer 3

Figure 4: FF hidden layer weights distribution

We then compared the validation accuracy over epochs for both models to see how well FF performs to BP, as shown in Figure 5. However due to the limitations of our code, we can see that the BP method was not able to achieve our original plan of comparing at a 98% test accuracy. We used the metric results for both models and calculated the ratio of FF over BP to see if the values will be *less than 1* (i.e. FF uses less resources than BP) or *greater than 1* (i.e. FF uses more resources than BP). Testing for statistical significance of our ratio results, we conducted a bootstrap method [Effron and Tibshirani, 1993] to see if our initial results were by chance.

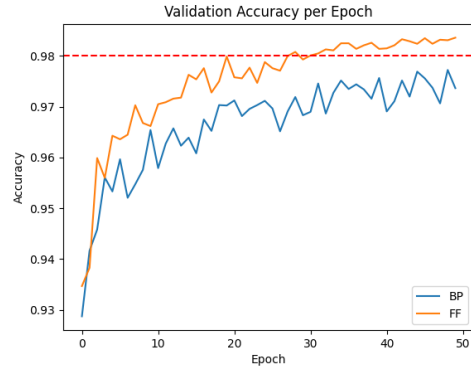
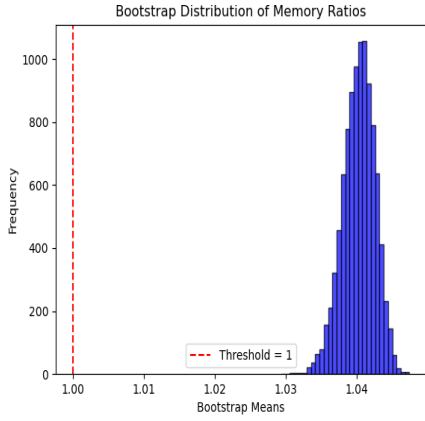
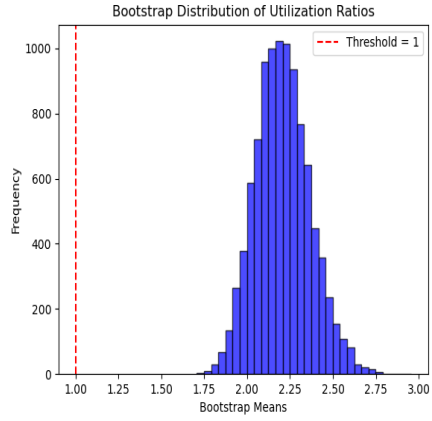


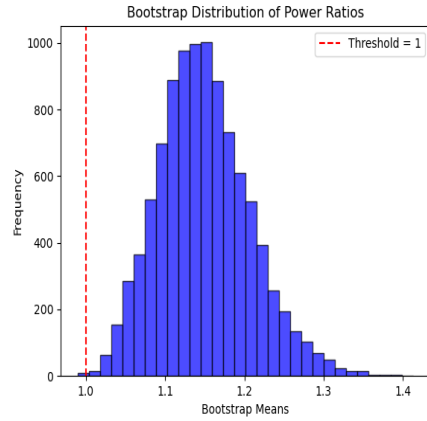
Figure 5: FF vs BP validation set accuracy



(a) Memory usage ratio



(b) Memory utilization ratio



(c) Power draw ratio

Figure 6: Bootstrapped metrics using 50 epochs

As shown in Figure 6, the same training iteration the FF method uses more resources than BP, in all three metric ratios. The p-value was 0 when conducting the test whether the FF/BP ratio is greater than 1 for power draw, memory utilisation

and memory usage. This contradicts what Hinton proposed in his paper; we were running the models without considering the time FF takes to achieve a similar generalisation power as BP.

Reflecting on the limitations (more details below) and our validation accuracy results, we adjusted our approach and conducted a new experiment control. In this experiment, we choose a threshold accuracy level that each model can achieve, and assessed the metrics they produced.

6.2 Test accuracy control

In the new experiment, we used test accuracy as the control. Each model was trained until it achieved a 97% test accuracy. We then compared the resource usage for each model to determine if there were significant differences.

Table 1 shows that on average FF consistently achieved the desired accuracy in less time than the BP model (50 seconds faster), used less average GPU compute, and less overall power. The difference in the memory usage was marginal, likely due to the model architectures being similar. The FF algorithm used 20% less total power than the BP algorithm to achieve the same level of accuracy. This significant difference suggests that the FF algorithm may offer some efficiency benefits compared with BP. The FF algorithm uses less GPU utilisation (27.39% vs 36.03% for BP), demonstrating that it may be better suited to situations where access to compute is limited such as embedded devices.

While there are caveats to this result (see limitations below), this does indicate that the FF algorithm provide advantages over the BP algorithm in this particular case.

Metric	BP		FF	
	Mean	Std	Mean	Std
Runtime (sec)	268.29	0.70	218.07	5.58
Power (watts)	19.36	0.60	19.41	2.63
Total Power Used (Joules)	5193.06	158.95	4238.42	621.09
Utilization (%)	36.03	0.49	27.39	1.60
Memory (MB)	1462.98	9.53	1466.69	6.03
Energy (kWh) ^a	0.001442	-	0.001177	-

Table 1: Model metrics comparison at threshold accuracy (97%)

^aCalculated as $\text{Energy(kWh)} = \frac{\text{Power}(w)}{1000} \times \frac{\text{Runtime}(sec)}{3600}$

6.3 Limitations

There were several challenges that limited our project approach.

Firstly, selecting the closest python implementation of Hinton’s code proved difficult. We initially chose Pezeshki and Yun’s code as it was relatively simpler than Löwex’s FF `Pytorch` code, but the former could not achieve a test accuracy higher than 96%⁶. The latter was able to achieve a higher test accuracy (around 98%) but required more time to comprehend and develop the additional code to visualise the results to compare with BP.

As a result, we did not have enough time to complete the other metrics and plots that we set out to do. We were also unable to compare the models on the CIFAR-10 dataset as initially planned. We now leave this as part of Future Research.

Another challenge this project faced was the inability to query all the intended GPU metrics from High Performance Computing (HPC) server as originally planned. To overcome this issue, an Nvidia Geforce GTX 1080 GPU was used instead. This is an older model card designed primarily for gaming purposes. This means the results from this project may not apply to newer GPU cards that use architectures designed for larger scale machine learning purposes. With that being said, the results from this project are still applicable to smaller scale operations utilising consumer grade cards.

Finally, the GPU monitoring code and supporting Python scripts may have affected model performance since they ran concurrently with each model. More rigorous testing methodology could yield more accurate results than those presented in this report.

This section is written by Kevin & Gabriel

7 Future Research

There are many other avenues of research we hope to continue from the project we conducted. Given the Forward-Forward algorithm is still in its early stages, it would be interesting to see the same metrics test conducted with the following modifications:

This section is written by Kevin & Gabriel

- Conducting a similar comparison test with other benchmark data sets (i.e. CIFAR-10, CIFAR-100, FMNIST) to see what metrics training on those datasets produce.
- Training with different network architectures. This project only worked on a fully connected net of 3 hidden layers with 1000 ReLUs. Future work could investigate the algorithm’s performance on:
 - Deeper neural networks
 - Convolutional neural networks (CNN)
 - Recurrent neural networks (RNN)
- Testing the FF algorithm on datasets outside of computer vision to determine how well it generalizes to other types of data. For example, how well the algorithm can be integrated with transformer models.

⁶for reference, the BP model could achieve accuracy of 98%

- Investigating parallelization techniques for the FF algorithm in relation to offline training, as hinted in Section 5 of Hinton’s paper.

8 Conclusion

In conclusion, the Forward-Forward algorithm is better suited when factors such as speed, power and memory is a constraint.

We have only scratched the surface of what can be said about the efficiency of the Forward-Forward algorithm as it pertains to the many different machine learning algorithms out there.

While it appears backpropagation will likely remain the dominate algorithm for training more complex neural networks, the Forward-Forward algorithm may find it’s niche in situations where energy usage is a constraint—a factor which is growing in importance as the global climate crisis worsens. Additionally, the Forward-Forward algorithm may offer advantages in decentralized computation and embedded devices as it is less demanding on computational resources.

Despite technical limitations during testing, our results indicate that the Forward-Forward algorithm remains a promising avenue for future research and potential improvements in machine learning techniques and efficiency.

This section is
written by
Gabriel

References

- S. Amari. A theory of adaptive pattern classifiers. *IEEE Transactions on Electronic Computers*, (3):299–307, 1967.
- A. Bewtra. A further investigation of the forward-forward algorithm. 2023. URL <https://aqbewtra.github.io/files/FF.pdf>.
- B. Efron and R. J. Tibshirani. An introduction to the bootstrap, 1993.
- S. Gandhi, R. Gala, J. Kornberg, and A. Sridhar. Extending the forward forward algorithm, 2023. URL <http://arxiv.org/abs/2307.04205>.
- I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/file/5ca3e9b122f61f8f06494c97b1afccf3-Paper.pdf.
- M. Gutmann and A. Hyvärinen. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pages 297–304. JMLR Workshop and Conference Proceedings, Mar. 2010. URL <https://proceedings.mlr.press/v9/gutmann10a.html>. ISSN: 1938-7228.
- G. Hinton. The forward-forward algorithm: Some preliminary investigations. URL <http://arxiv.org/abs/2212.13345>.
- G. E. Hinton and T. J. Sejnowski. Learning and Relearning in Boltzmann Machines. In M. I. Jordan and T. J. Sejnowski, editors, *Graphical Models*, pages 45–76. The MIT Press, Oct. 2001. ISBN 978-0-262-29120-0. doi: 10.7551/mitpress/3349.003.0005. URL <https://direct.mit.edu/books/book/4345/chapter/181964/Learning-and-Relearning-in-Boltzmann-Machines>.
- Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. 521(7553):436–444. ISSN 1476-4687. doi: 10.1038/nature14539. URL <https://www.nature.com/articles/nature14539>. Publisher: Nature Publishing Group.
- Y. LeCun, C. Cortes, and C. Burges. Mnist handwritten digit database. *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, 2, 2010.
- T. P. Lillicrap, A. Santoro, L. Marris, C. J. Akerman, and G. Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.
- A. Momeni, B. Rahmani, M. Malléjac, P. d. Hougne, and R. Fleury. PhyFF: Physical forward forward algorithm for in-hardware training and inference. 2023. URL <https://openreview.net/forum?id=aqsFLVg0tY>.
- D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia. Efficient large-scale language model training on GPU clusters using megatron-LM, 2021. URL <http://arxiv.org/abs/2104.04473>.

- H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian. A comprehensive overview of large language models, 2024. URL <https://arxiv.org/abs/2307.06435>.
- D. Paliotta, M. Alain, B. Máté, and F. Fleuret. Graph neural networks go forward-forward, 2023. URL <http://arxiv.org/abs/2302.05282>.
- A. Reyes-Angulo and S. Paheding. The forward-forward algorithm as a feature extractor for skin lesion classification: A preliminary study, 2023. URL <http://arxiv.org/abs/2307.00617>.
- F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. ISSN 1939-1471. doi: 10.1037/h0042519. Place: US Publisher: American Psychological Association.
- D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. 323(6088):533–536, 1986. ISSN 1476-4687. doi: 10.1038/323533a0. URL <https://www.nature.com/articles/323533a0>. Publisher: Nature Publishing Group.
- R. Scodellaro, A. Kulkarni, F. Alves, and M. Schröter. Training convolutional neural networks with the forward-forward algorithm, 2024. URL <http://arxiv.org/abs/2312.14924>.
- L. Van der Maaten and G. Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

9 Appendix

	BP Runtime	FF Runtime
0	268.55	219.01
1	267.35	218.90
2	266.76	225.31
3	268.49	220.16
4	269.16	220.59
5	268.43	207.77
6	268.57	222.45
7	268.40	208.76
8	268.62	219.90
9	268.56	217.88

Table 2: Training Runtime Across 10 Runs

	Average BP Memory	Average FF Memory
0	1474.50	1480.57
1	1485.22	1472.01
2	1462.65	1462.94
3	1458.68	1459.93
4	1457.58	1463.05
5	1461.80	1462.52
6	1460.06	1467.57
7	1458.03	1468.28
8	1455.09	1466.93
9	1456.20	1463.12

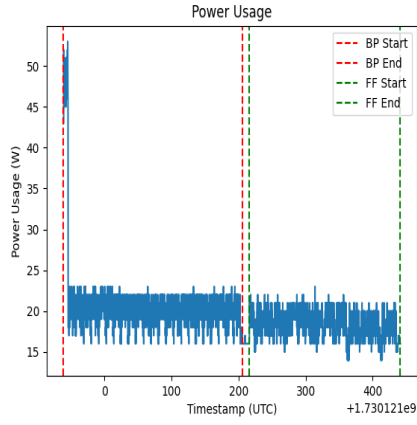
Table 3: GPU Memory Usage Across 10 Runs

	Average BP Utilization	Average FF Utilization
0	36.88	28.27
1	36.98	29.66
2	35.73	27.64
3	35.81	27.96
4	35.87	24.44
5	35.72	28.17
6	35.90	24.72
7	35.64	27.84
8	35.89	27.36
9	35.85	27.83

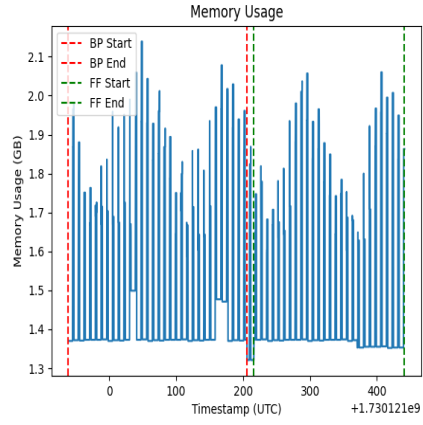
Table 4: GPU Utilization Across 10 Runs

	Average BP Power	Average FF Power	Total BP Power	Total FF Power
0	20.59	23.12	5530.01	5064.11
1	20.38	18.24	5448.86	3992.95
2	19.07	17.78	5086.19	4006.09
3	19.09	17.63	5125.77	3881.84
4	19.20	23.43	5168.84	5169.26
5	19.01	17.61	5103.74	3659.06
6	19.10	23.08	5128.57	5134.20
7	18.98	17.63	5095.42	3680.97
8	19.06	17.85	5120.42	3924.24
9	19.08	17.77	5122.76	3871.49

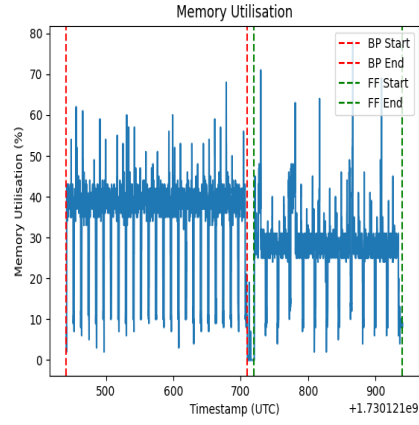
Table 5: GPU Power Draw Across 10 Runs



(a) Power Usage over time



(b) Memory utilization over time



(c) GPU Utilization over time

Figure 7: Metrics over time